

A Project Report on

“Smart Expenses Tracker”

SUBMITTED BY

Mr. Aryan Santosh Lavate

Mr. Umarriyan Azharhusaini Patil

UNDER THE GUIDE OF

Mr. Karande V. D. Sir

Ms. Mujawar P.S. Mam

SUBMITTED TO

Punyashlok Ahilyadevi Holkar Solapur University, Solapur

DEPARMENT OF

Bachelor of Computer Applications



**Deshbhakta Sambhajirao Garad Mahavidyalaya,
Mohol.**

Academic Year: -2025-2026



Deshbhakta Sambhajirao Garad Mahavidyalaya, Mohol.

CERTIFICATE

This is Certify that

Mr. Aryan Santosh Lavate

Mr. Umarriyan Azharhusaini Patil

Have Successfully completed Project work entitled,

“Smart Expenses Tracker”

Has been submitted by of T.Y (B.C.A) Final in partial fulfilment of the requirements for the award of Degree Bachelor of Computer Applications as per curriculum laid by the university during academic year 2025-2026.

Ms. Mujawar P.S

(Project Guide)

Mr. Karande V.D

(H.O.D)

Patil N.J

(Principal)

Examiner

Date: / /2026



Deshbhakta Sambhajirao Garad Mahavidyalaya, Mohol.

CERTIFICATE

This is Certify that

Mr. Aryan Santosh Lavate

Have Successfully completed Project work entitled,

“Smart Expenses Tracker”

Has been submitted by of T.Y (B.C.A) Final in partial fulfilment of the requirements for the award of Degree Bachelor of Computer Applications as per curriculum laid by the university during academic year 2025-2026.

Ms. Mujawar P.S

(Project Guide)

Mr. Karande V.D

(H.O.D)

Patil N.J

(Principal)

Examiner

Date: / /2026



Deshbhakta Sambhajirao Garad Mahavidyalaya, Mohol.

CERTIFICATE

This is Certify that

Mr. Umarriyan Azharhusaini Patil

Have Successfully completed Project work entitled,

“Smart Expenses Tracker”

Has been submitted by of T.Y (B.C.A) Final in partial fulfilment of the requirements for the award of Degree Bachelor of Computer Applications as per curriculum laid by the university during academic year 2025-2026.

Ms. Mujawar P.S

(Project Guide)

Mr. Karande V.D

(H.O.D)

Patil N.J

(Principal)

Examiner

Date: / /2026

Declaration

I hereby declare that the [Smart Expenses Tracker](#) Project is an original work completed by me and represents my own ideas and efforts. Wherever the ideas, data, or words of others have been used, they have been properly cited and acknowledged. I affirm that I have followed the principles of academic honesty and integrity throughout the development of this project. I confirm that no information, data, or results presented in this project have been fabricated, falsified, or misrepresented. This submission has been prepared solely for academic purposes and reflects truthful work to the best of my knowledge. I understand that any violation of academic rules or ethical standards may result in disciplinary action as per the norms and regulations of the institution.

ACKNOWLEDGEMENT

We begin by expressing our sincere gratitude to the Almighty for providing us with the strength, determination, and wisdom to successfully complete our project titled “[Smart Expenses Tracker](#).” This project has been a valuable learning experience and would not have been possible without proper guidance and encouragement. We would like to convey our heartfelt thanks to our project guide, **Mrs.Mujawar mam**, for their continuous guidance, insightful suggestions, and constant motivation throughout the development of this project. Their expertise and support played a crucial role in our ideas and helping us overcome challenges at every stage.

We also extend our deep appreciation to **Mr. Karande Sir, Head of the Department**, for providing moral support and creating an encouraging academic environment that enabled us to work efficiently. Our sincere thanks go to Mr. Patil Sir, Principal of our institution, for granting us access to the necessary resources and facilities required for the successful completion of this project.

Finally, we are truly thankful to our friends and family members for their unwavering support, encouragement, and understanding throughout the course of this work.

Thank you.

Mr. Aryan Santosh Lavate

Mr. Umarriyan Azharhusaini Patil

INDEX

Sr. No.	Title	Page No.
1	CHAPTER 1: INTRODUCTION	1
	1.1. Existing System and Need for System	2
	1.2. Scope of Work	3
	1.3. Operating Environment – Hardware and Software	4
	1.4. Detail Description of Technology Used	5
2	CHAPTER 2: PROPOSED SYSTEM	
	2.1. Proposed System	10
	2.2. Objectives of System	11
	2.3. User Requirements	12
3	CHAPTER 3: ANALYSIS & DESIGN	
	3.1. Data Flow Diagram (DFD)	13
	3.2. Entity Relationship Diagram (ERD)	14
	3.3. Data Dictionary	15
	3.4. Table Design	18
	3.5. Code Design	21
	3.6. Menu Tree	25
	3.7. Input Screens	26
	3.8. Report Formats	39
	3.9. Test Procedures and Implementation	41
4	CHAPTER 4: USER MANUAL	
	4.1. User Manual	47
	4.2. Operations Manual / Menu Explanation	49
	4.3. Forms and Report Specifications	52
	4.4. Drawbacks and Limitations	57
	4.5. Proposed Enhancements	58
5	CHAPTER 5: Conclusions	59
6	Bibliography	60

Sr. No.	Title	Page No.
7	ANNEXURES	
	7.1. ANNEXURE 1: INPUT FORMS WITH DATA	61
	7.2. ANNEXURE: SAMPLE CODE	65

CHAPTER 1: INTRODUCTION

In today's fast-paced digital world, effective financial management has become essential for individuals and households. Many people struggle to keep track of their daily expenses, leading to poor budgeting and unnecessary spending. Traditional methods such as manual record keeping or spreadsheets are time-consuming and prone to errors.

The Smart Expenses Tracker is designed to provide a simple, efficient, and intelligent solution for managing personal finances. This system helps users record, categorize, and analyse their expenses in real time. By offering automated tracking, visual reports, and budget insights, the application enables users to make informed financial decisions and develop better spending habits. The project aims to simplify expense management through a user-friendly and reliable digital platform.

1.1. Existing System and Need for System

Existing System

The existing expense tracking methods are mostly manual or partially digital. Many users maintain their expenses using notebooks, spreadsheets, or basic mobile applications. Manual systems are time-consuming, error-prone, and difficult to manage over long periods. Spreadsheet-based systems require continuous manual input and lack automation and intelligent analysis. Existing applications often provide limited features and do not offer meaningful insights into spending patterns, making effective financial planning difficult.

Need for the System

There is a need for a smart and automated expense tracking system to overcome the limitations of existing methods. The **Smart Expenses Tracker** is designed to help users easily record, categorize, and monitor their expenses in real time. The system provides better organization, analysis, and visualization of expenses, enabling users to understand their spending habits and manage their finances efficiently. This system reduces manual effort, minimizes errors, and supports effective budgeting and financial decision-making.

1.2 Scope of the System

The **Smart Expenses Tracker** is designed to provide an efficient and user-friendly platform for managing personal financial expenses. The system focuses on helping users record, monitor, and analyze their daily spending in an organized and structured manner.

The scope of the system includes:

- Allowing users to securely register and log in to the application.
- Enabling users to add, edit, and delete expense records.
- Categorizing expenses such as food, travel, shopping, etc.
- Providing expense summaries.
- Generating graphical reports for better visualization of spending patterns.
- Allowing users to compare actual expenses with planned budgets.
- Ensuring secure data storage and authentication using modern security mechanisms.
- Providing a responsive web-based interface accessible through browsers.

1.3. Operating Environment – Hardware and Software

The **Smart Expenses Tracker** system is designed to operate in a standard computing environment and does not require high-end hardware, making it accessible to a wide range of users.

Hardware Requirements:

- Processor: Intel / Ryzen 3 or higher
- RAM: Minimum 4 GB
- Hard Disk: Minimum 10 GB free space
- Input Devices: Keyboard, Mouse
- Output Devices: Monitor
- Internet connection.

Software Requirements:

- Operating System: Windows 10
- Browser: Google Chrome / Firefox / Any Other

1.4. Detail Description of Technology Used

Frontend:

- **Library:** React

React is a component-based JavaScript library used to build dynamic and interactive user interfaces. It enables efficient UI updates through the Virtual DOM and supports reusable components for better code maintainability. React also simplifies state management and improves application performance.

- **Build Tool:** Vite

Vite is a modern frontend build tool that provides extremely fast development server startup and optimized production builds. It uses native ES modules during development, enabling faster hot module replacement (HMR). Vite significantly improves development speed and overall project efficiency.

- **Styling:** Tailwind CSS

Tailwind CSS is a utility-first CSS framework used to create modern and responsive user interfaces. It allows developers to apply styling directly through predefined utility classes, reducing the need for custom CSS files. Tailwind enhances design consistency and speeds up UI development.

- **Routing:** React Router

React Router is used to implement client-side routing within the application. It enables seamless navigation between different pages such as Home, Insurance, Reports, Features, About, and User Guide without reloading the page. This improves user experience and maintains application state during navigation.

- **HTTP Client:** Axios

Axios is a promise-based HTTP client used to communicate with backend APIs. It handles GET, POST, PUT, and DELETE requests efficiently and supports request/response interceptors. In this project, Axios is configured to automatically attach JWT tokens to secure API requests.

- **Icons:** Lucide React

Lucide React provides lightweight, customizable, and scalable SVG icons for enhancing the user interface. The library ensures consistent icon styling across the application. It improves visual clarity and user interaction through intuitive graphical symbols.

- **Charts:** Recharts

Recharts is a composable charting library built specifically for React applications. It is used to display expense analytics through bar charts, line graphs, and pie charts. The library helps users visualize financial trends and spending patterns effectively.

- **Utility Libraries:**

The `clsx` library is used to conditionally combine CSS class names based on component state. The `tailwind-merge` library resolves class conflicts in Tailwind CSS by intelligently merging utility classes. Together, they ensure cleaner and more maintainable styling logic.

Backend:

- **Programming Language:** Python

Python is used for implementing server-side logic and handling API requests. It provides simplicity, readability, and a vast ecosystem of libraries. Python enables rapid backend development and efficient integration with databases and authentication mechanisms.

- **Framework:** FastAPI

FastAPI is a high-performance web framework used to develop RESTful APIs. It provides automatic API documentation through Swagger UI and Redoc. FastAPI supports asynchronous programming, improving response time and scalability of the application.

- **Server:** Uvicorn

Uvicorn is an ASGI (Asynchronous Server Gateway Interface) server used to run the FastAPI application. It supports asynchronous request handling and ensures high-speed performance. Uvicorn efficiently manages incoming HTTP requests and API responses.

- **ORM:** SQLAlchemy

SQLAlchemy is an Object Relational Mapper (ORM) that maps Python classes to database tables. It allows developers to perform database operations using Python objects instead of writing raw SQL queries. SQLAlchemy improves code readability, database abstraction, and data integrity.

- **Authentication & Security:**

- **JWT (JSON Web Tokens) using python-jose**

JWT is used for secure user authentication and authorization. Upon successful login, a token is generated and attached to API requests to verify user

identity. The python-jose library is used to encode and decode JWT tokens securely.

- **Passlib (bcrypt)**

Passlib with bcrypt hashing algorithm is used to securely store user passwords. Instead of saving plain text passwords, hashed values are stored in the database. This ensures enhanced protection against unauthorized access and data breaches.

- **Data Validation:**

- **Pydantic**

Pydantic is used for request and response validation in FastAPI. It ensures that incoming data follows the defined schema and data types. This improves reliability, prevents invalid data entry, and enhances API consistency.

Database:

- **Database Management System: MySQL**

MySQL is used as the relational database management system for storing application data. It maintains structured tables for users, transactions, insurance policies, and financial records. MySQL ensures data integrity, reliability, and efficient query performance.

- **Database Connector: pymysql**

PyMySQL is a Python-based MySQL client used to establish communication between the FastAPI backend and the MySQL database. It enables execution of database queries and supports secure connection handling.

- **ORM Support: SQLAlchemy**

SQLAlchemy integrates with MySQL to manage database operations efficiently. It handles table relationships, constraints, and transactions. This ensures clean database design and simplifies CRUD operations within the application.

CHAPTER 2: PROPOSED SYSTEM

2.1. Proposed System

The **Smart Expenses Tracker** is a web-based application designed to help users efficiently record, manage, and analyse their daily expenses. The proposed system overcomes the limitations of traditional and existing expense management methods by providing an automated, secure, and user-friendly platform.

The system allows users to register and log in securely using authentication mechanisms. Once logged in, users can add their daily expenses by specifying details such as amount, category, date, and description. All expense data is stored securely in a centralized database, ensuring data integrity and easy retrieval.

The proposed system provides automatic categorization of expenses and generates visual reports such as charts and summaries to help users understand their spending patterns. Users can monitor monthly and yearly expenses, compare spending across categories, and set budgets to control unnecessary expenditures. Alerts and insights help users make informed financial decisions.

The backend of the system is implemented using **FastAPI**, which handles API requests, business logic, authentication, and database communication. The frontend is developed using **React**, providing a responsive and interactive user interface. **MySQL** is used as the database to store user information and expense records in a structured manner.

Overall, the proposed system offers an efficient, scalable, and reliable solution for personal expense management, enabling users to track finances effectively, improve budgeting habits, and achieve better financial planning.

2.2. Objectives of System

The main objectives of the **Smart Expenses Tracker** are as follows:

- To provide a simple and user-friendly platform for recording their expenses.
- To automate the tracking and management of personal expenses.
- To categorize expenses for better understanding of spending patterns.
- To generate graphical reports and summaries for expense analysis.
- To help users plan budgets and control unnecessary spending.
- To ensure secure storage and management of user financial data.
- To reduce manual effort and minimize errors in expense tracking.
- To support better financial decision-making and money management.

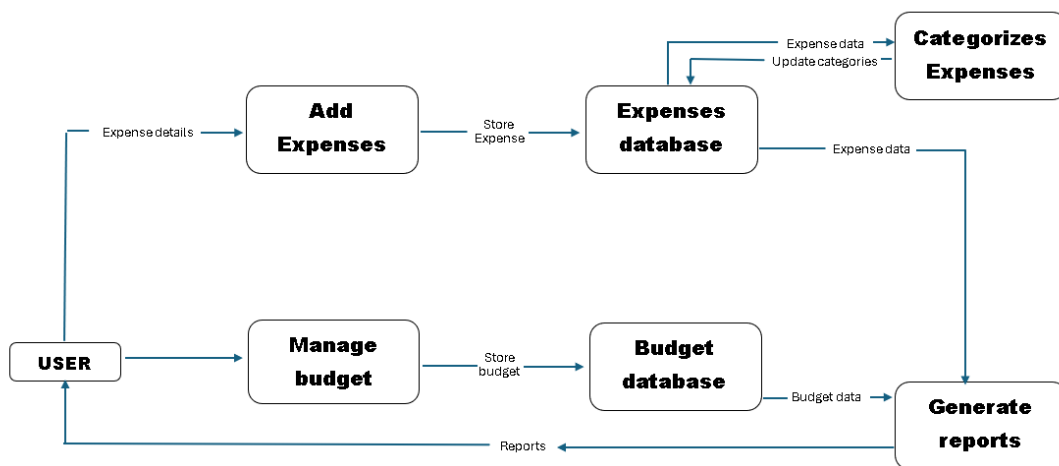
2.3. User Requirements

The **Smart Expenses Tracker** system is designed to meet the following user requirements:

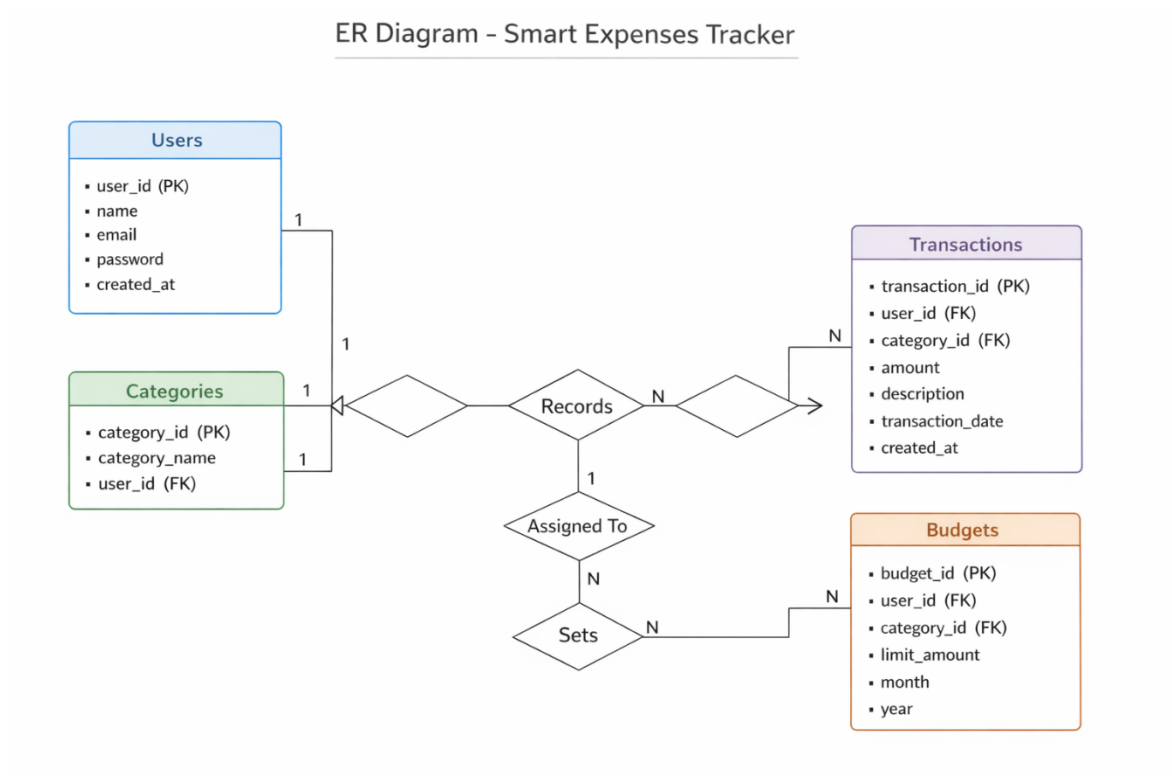
- The user should be able to register and log in securely to the system.
- The system should allow users to add, update, and delete expense records.
- Users should be able to categorize expenses such as food, travel, shopping, and utilities.
- The system should display expenses in a clear and organized manner.
- Users should be able to view expense summaries.
- The system should generate graphical representations such as charts for expense analysis.
- The system should provide secure storage of user data and expense information.
- The application should be accessible through a web browser with a user-friendly interface.
- The system should provide fast response time and reliable performance.

CHAPTER 3: ANALYSIS & DESIGN

3.1. Data Flow Diagram (DFD)



3.2. Entity Relationship Diagram (ERD)



3.3. Data Dictionary

User Table:

Column Name	Data Type	Key	Null	Default	Description
Id	INT	PK	No	Auto Increment	Unique identifier for each user
Email	VARCHAR (255)	UQ	No	NULL	Email address of the user (must be unique)
hashed_password	VARCHAR (255)	—	No	NULL	Encrypted password of the user
full_name	VARCHAR (255)	—	Yes	NULL	Full name of the user
is_active	TINYINT (1)	—	Yes	NULL	Indicates whether the user account is active (1 = Active, 0 = Inactive)

Transaction Table:

Column Name	Data Type	Key	Null	Default	Description
Id	INT	PK	No	Auto Increment	Unique identifier for each transaction
user_id	INT	FK	No	NULL	References the user who performed the transaction
amount	FLOAT	—	Yes	NULL	Transaction amount
category	VARCHAR (255)	—	Yes	NULL	Category of transaction (e.g., Food, Rent, Salary)
Type	VARCHAR (50)	—	Yes	NULL	Type of transaction (Income / Expense)
description	VARCHAR (255)	—	Yes	NULL	Additional details about the transaction
Date	DATETIME	—	Yes	CURRENT_TIMESTAMP	Date and time of transaction

Insurance Table:

Column Name	Data Type	Key	Null	Default	Description
id	INT	PK	No	Auto Increment	Unique identifier for each insurance policy
user_id	INT	FK	No	—	References the user who owns the policy
provider	VARCHAR (255)	—	No	—	Name of the insurance provider company
type	VARCHAR (50)	—	No	—	Type of insurance (e.g., Health, Life, Vehicle)
policy_number	VARCHAR (100)	—	No	—	Unique policy identification number
premium	FLOAT	—	No	—	Premium amount to be paid
coverage	FLOAT	—	No	—	Total coverage amount of the policy
expiry_date	DATE	—	No	—	Policy expiration date
created_at	DATETIME	—	No	CURRENT_TIMESTAMP	Date and time when record was created
updated_at	DATETIME	—	No	CURRENT_TIMESTAMP	Date and time when record was last updated

3.4. Table Design

Users table:

users · Table



Table Name: Schema: **expense_tracker**

Charset/Collation: Engine:

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
 id	INT	☒	☒	☐	☐	☐	☐	☒	☐	
 email	VARCHAR(255)	☐	☐	☒	☐	☐	☐	☐	☐	NULL
 hashed_password	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
 full_name	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
 is_active	TINYINT(1)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
 role	VARCHAR(50)	☐	☐	☐	☐	☐	☐	☐	☐	NULL

Transactions table:

insurance_policies - Table transactions - Table users - Table

Table Name: Schema: **expense_tracker**

Charset/Collation: Engine:

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
id	INT	☒	☒	☐	☐	☐	☐	☒	☐	
user_id	INT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
amount	FLOAT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
category	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
type	VARCHAR(50)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
description	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
date	DATETIME	☐	☐	☐	☐	☐	☐	☐	☐	now()

User table:

insurance_policies - Table transactions - Table **users - Table** ✕

 Table Name: Schema: **expense_tracker**

Charset/Collation: Engine:

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
 id	INT	☒	☒	☐	☐	☐	☐	☒	☐	
 email	VARCHAR(255)	☐	☐	☒	☐	☐	☐	☐	☐	NULL
 hashed_password	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
 full_name	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
 is_active	TINYINT(1)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
		☐	☐	☐	☐	☐	☐	☐	☐	

3.5. Code Design

The Smart Expenses Tracker follows a **modular and layered code design**, where frontend, backend, and database logic are clearly separated. This structure improves maintainability, scalability, and ease of development.

Backend Code Design (FastAPI – Python)

Root Level

- **main.py**
 - Entry point of the backend application.
 - Initializes the FastAPI app.
 - Includes all API routers from the app/api/endpoints directory.
 - Configures middleware and application settings.

app/api/endpoints/

- Contains API route files such as:
 - auth.py
 - transactions.py
- Each file defines REST API endpoints and handles request routing.
- Endpoints call database and business logic functions.

app/models/

- Contains SQLAlchemy models:
 - user.py
 - transaction.py
- Each model represents a database table.
- Defines table relationships and constraints.

app/schemas/

- Contains Pydantic schemas for request and response validation.
- Ensures structured data exchange between frontend and backend.
- Used for serializing database objects into JSON responses.

app/db/

- Handles database configuration and connections.
- Includes:
 - Database session management
 - Engine configuration (MySQL using pymysql)

app/core/

- Contains core application utilities:
 - JWT authentication logic
 - Security configurations
 - Password hashing utilities
 - Environment settings

Frontend Code Design (React – Vite)

Frontend/ dist

Contains the entire user interface logic.

frontend/src/components/

- Contains reusable UI components:
 - Admin
 - Health Scor
 - Forms
 - Tables
 - Charts
 - Modals
 - Theme

pages/

- Contains application pages:
 - About
 - Admin
 - Dashboard
 - FD page
 - Feature page
 - Landing page
 - Login page
 - Register page
 - Report page

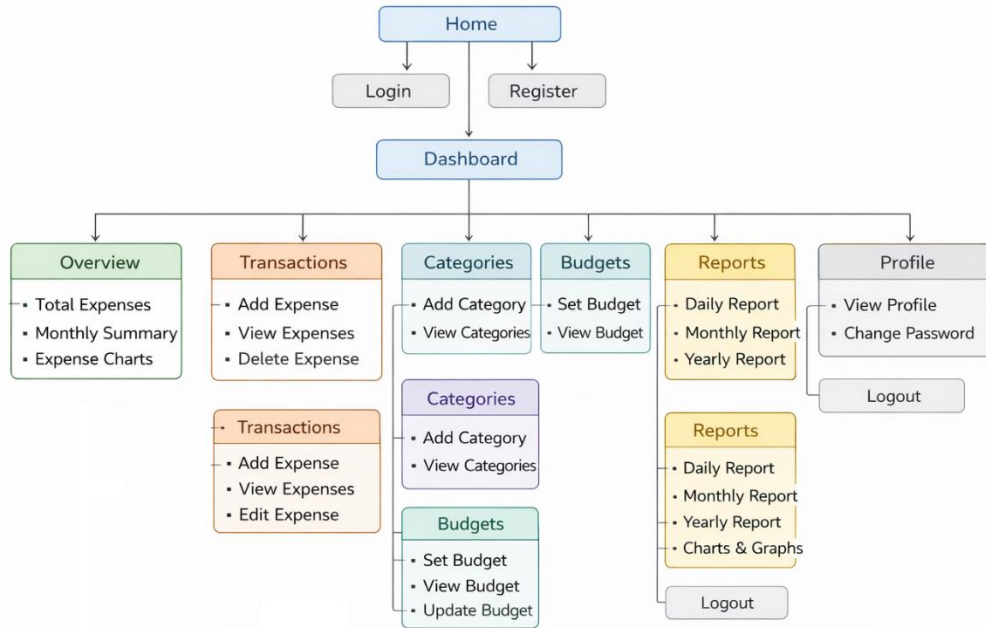
- Transaction page
- User guide page

services/

- Contains API service files:
 - api.js
 - insurance.js
 - transaction.js
- Uses Axios for HTTP requests.
- Handles JWT token attachment using interceptors.

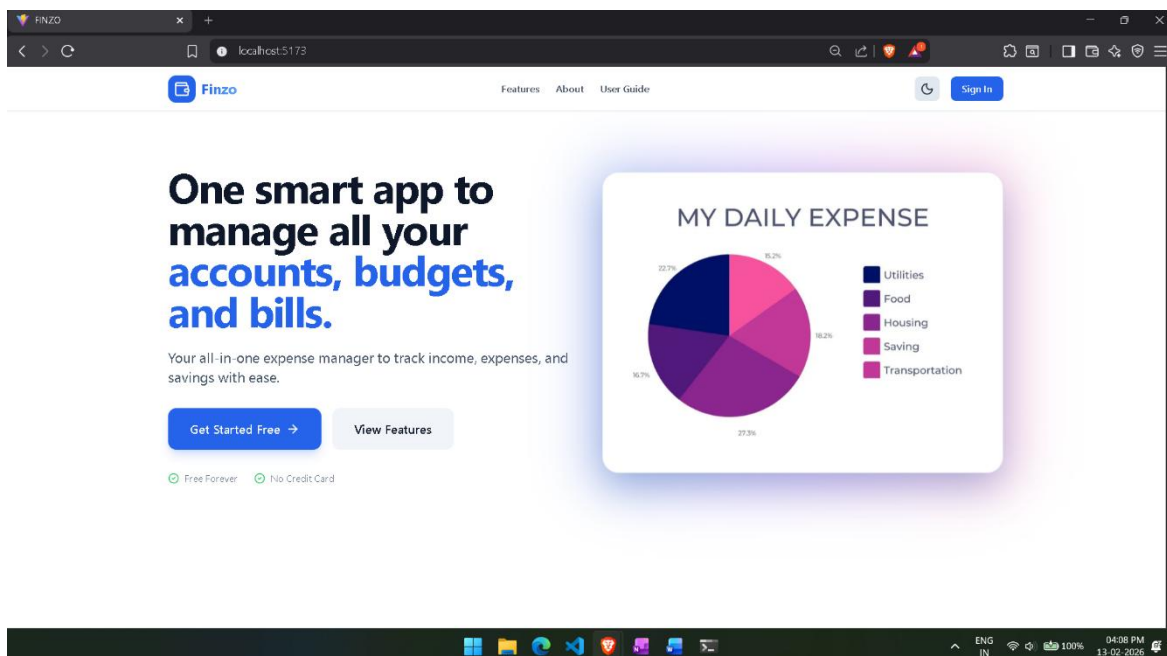
3.6. Menu Tree

Menu Tree - Smart Expenses Tracker

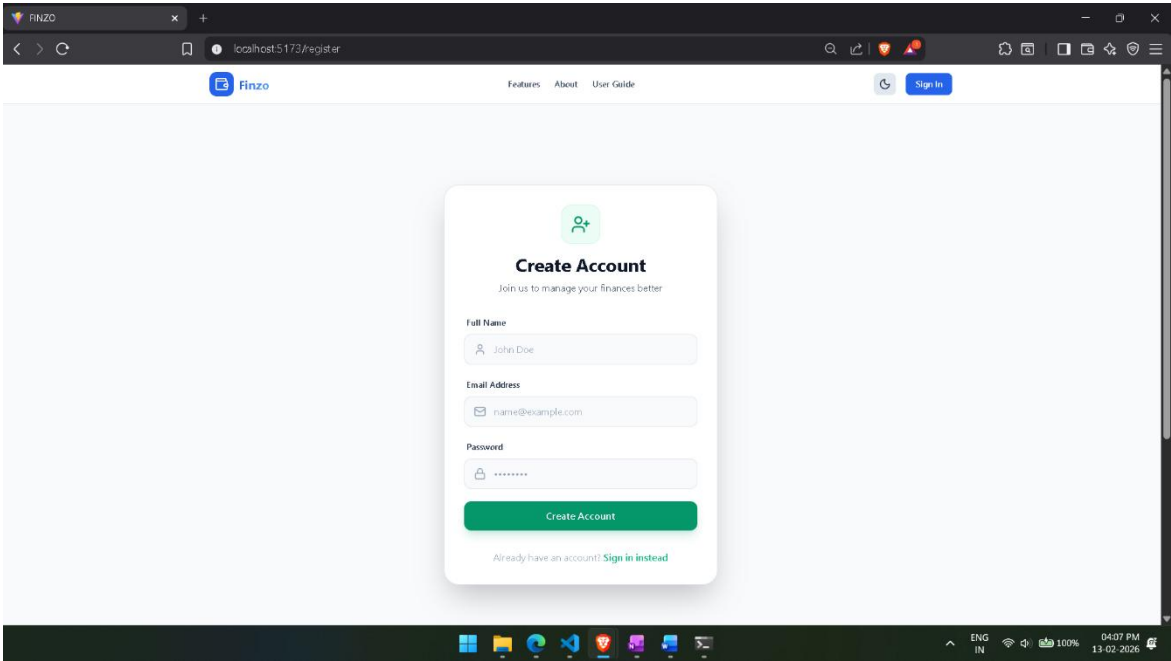


3.7. Input Screens

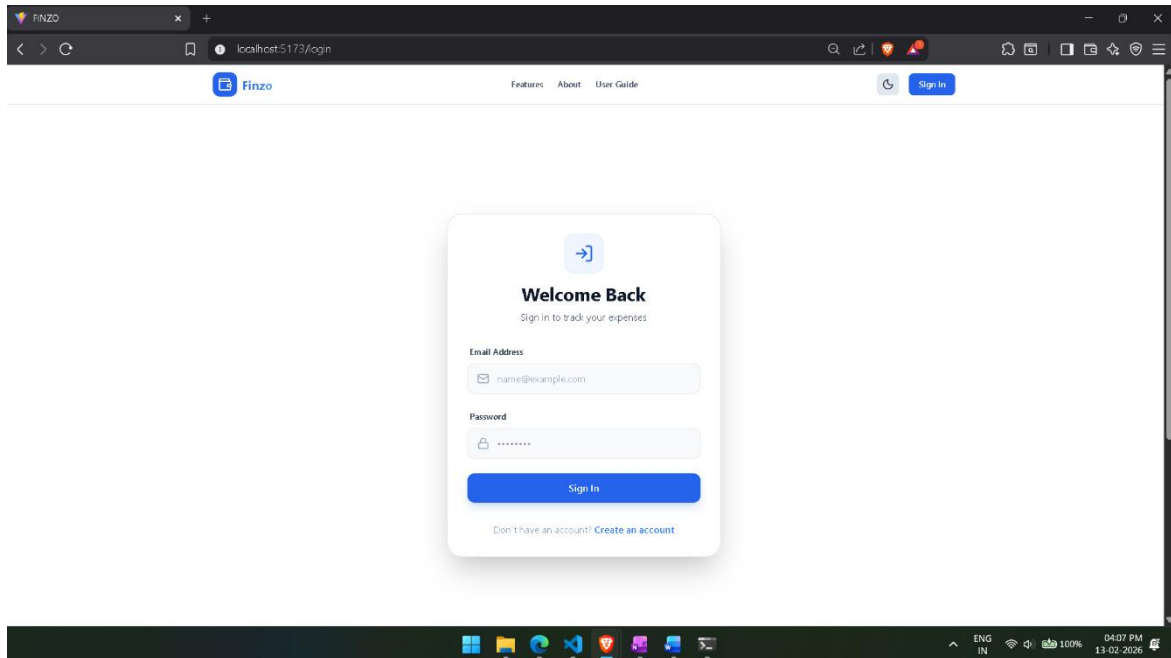
Landing page/Dashboard:



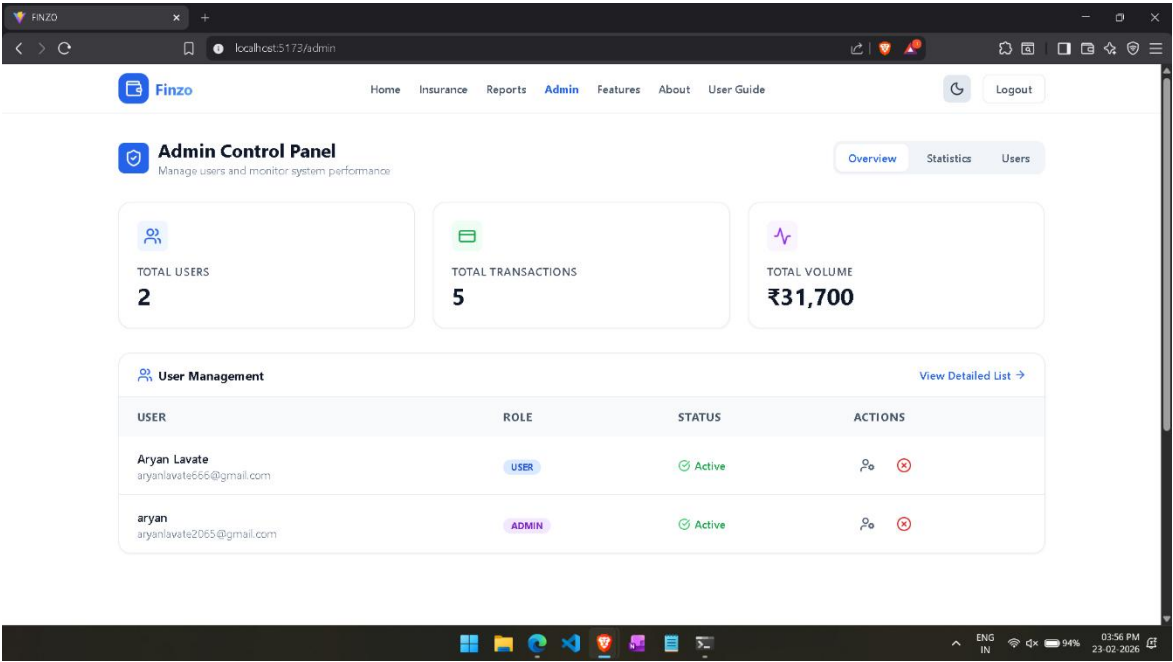
Register Page:



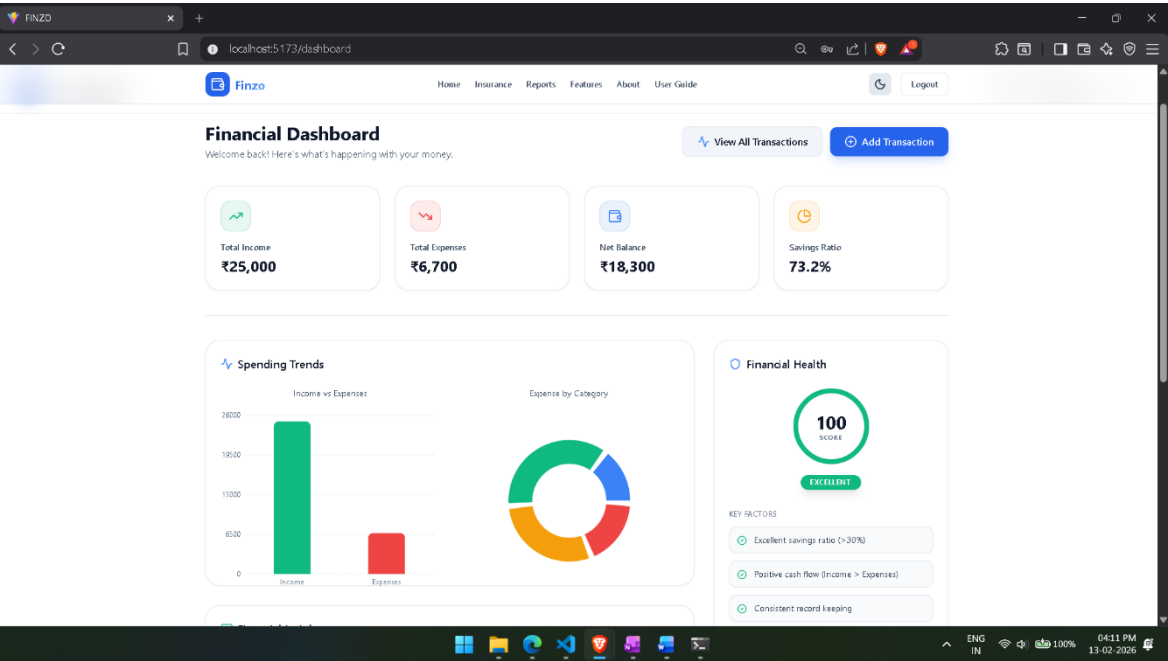
Login page:



Admin Panel:

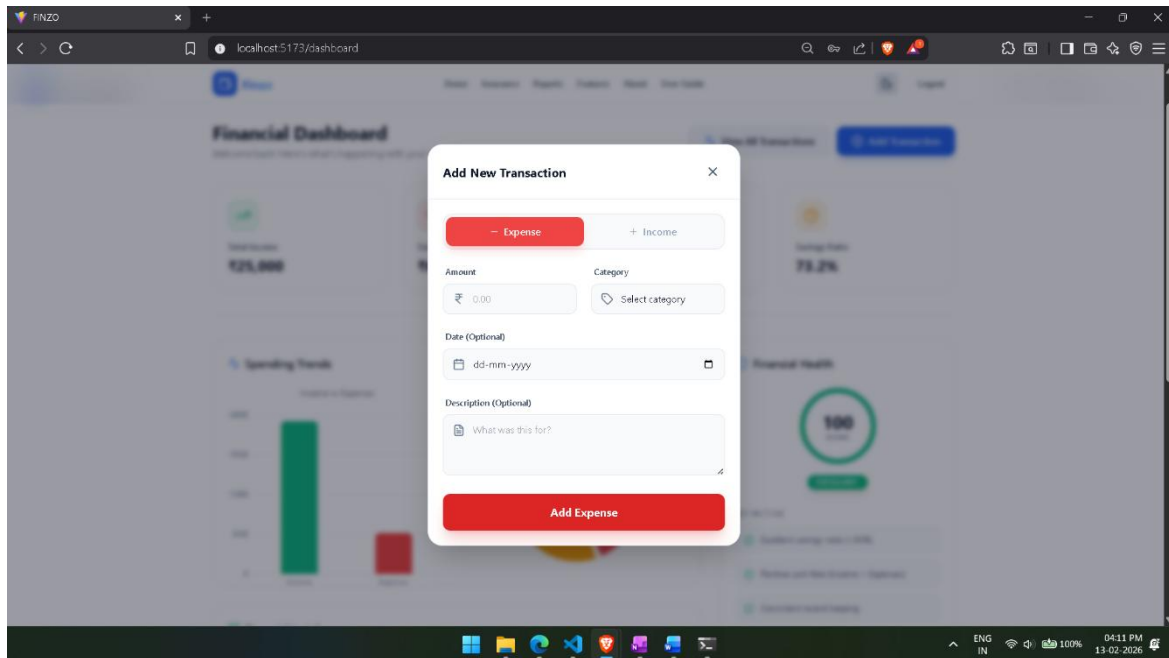


Home Page (Financial dashboard):

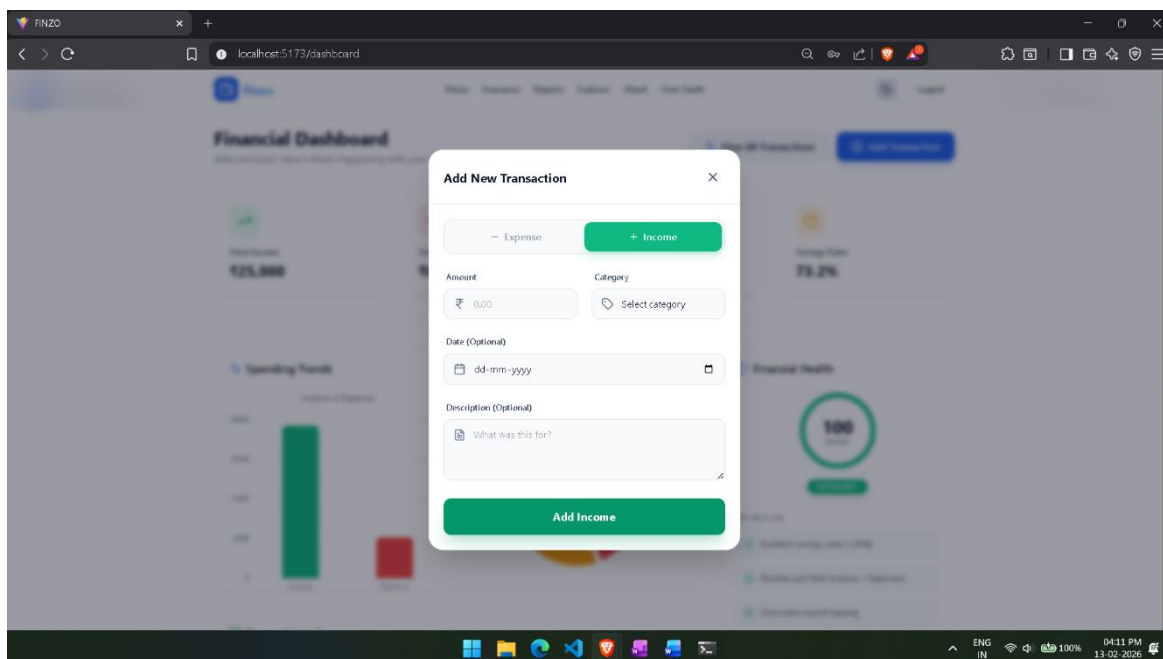


Add Transactions Page:

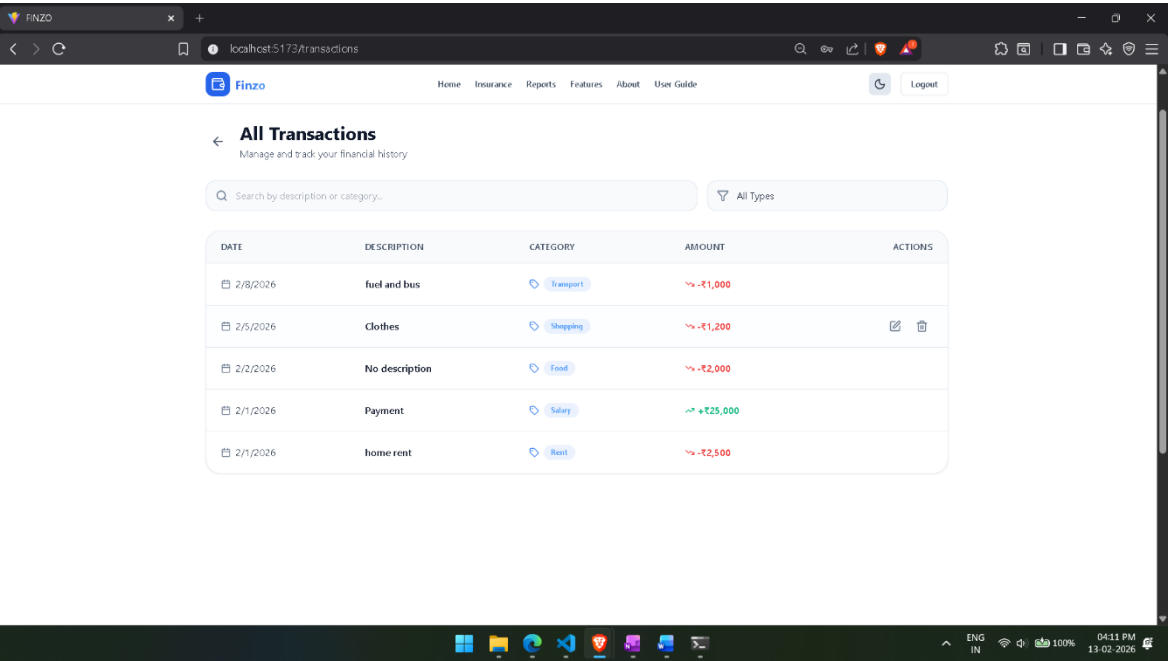
Add Expense:



Add Income:



All Transactions Page:



Insurance Page:

The screenshot displays the 'Insurance Overview' page of the Finzo application. The page has a dark header with the Finzo logo and navigation links: Home, Insurance, Reports, Features, About, and User Guide. A 'Logout' button is in the top right. The main content area is titled 'Insurance Overview' with the subtitle 'Track your health, vehicle and other insurance policies in one place.'.

On the left, there is a form titled 'Add Insurance Policy' with the following fields:

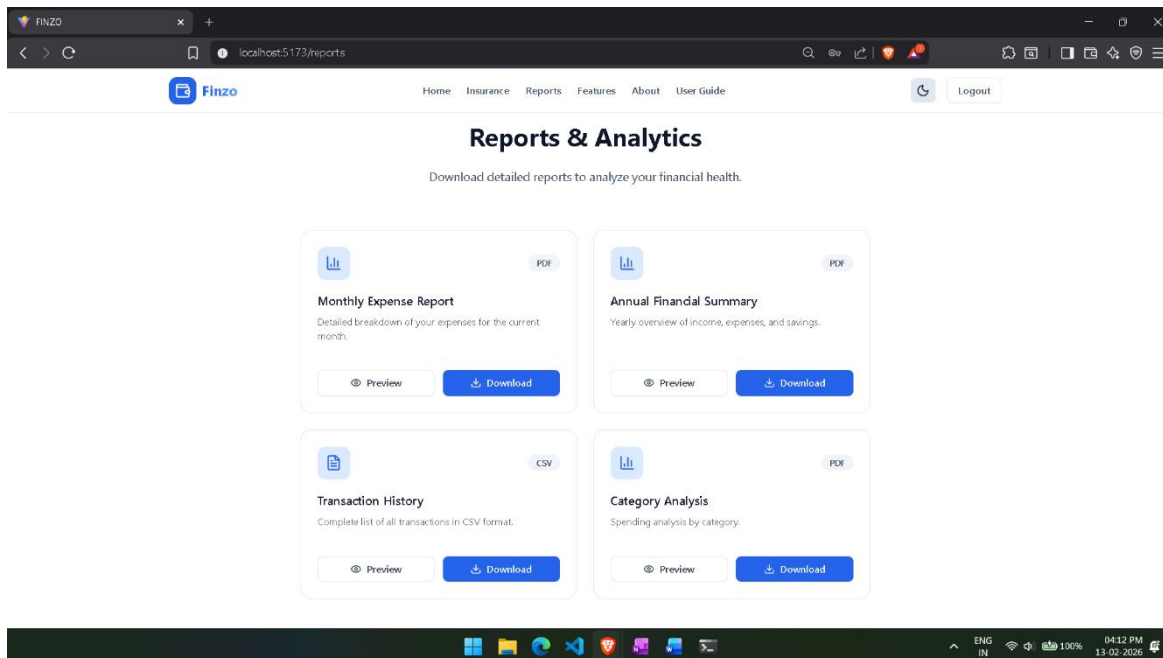
- Provider / Company:** A text input field with a placeholder 'e.g. LIC, HDFC Ergo'.
- Insurance Type:** A dropdown menu currently showing 'Health'.
- Policy Number:** A text input field with a placeholder 'e.g. POL12345678'.
- Annual Premium (₹):** A text input field with a placeholder 'e.g. 12000'.
- Coverage Amount (₹):** A text input field with a placeholder 'e.g. 500000'.
- Expiry / Renewal Date:** A date picker field showing 'dd-mm-yyyy'.
- Save Policy:** A blue button at the bottom of the form.

On the right, there is a section titled 'Your Policies' containing a single policy card:

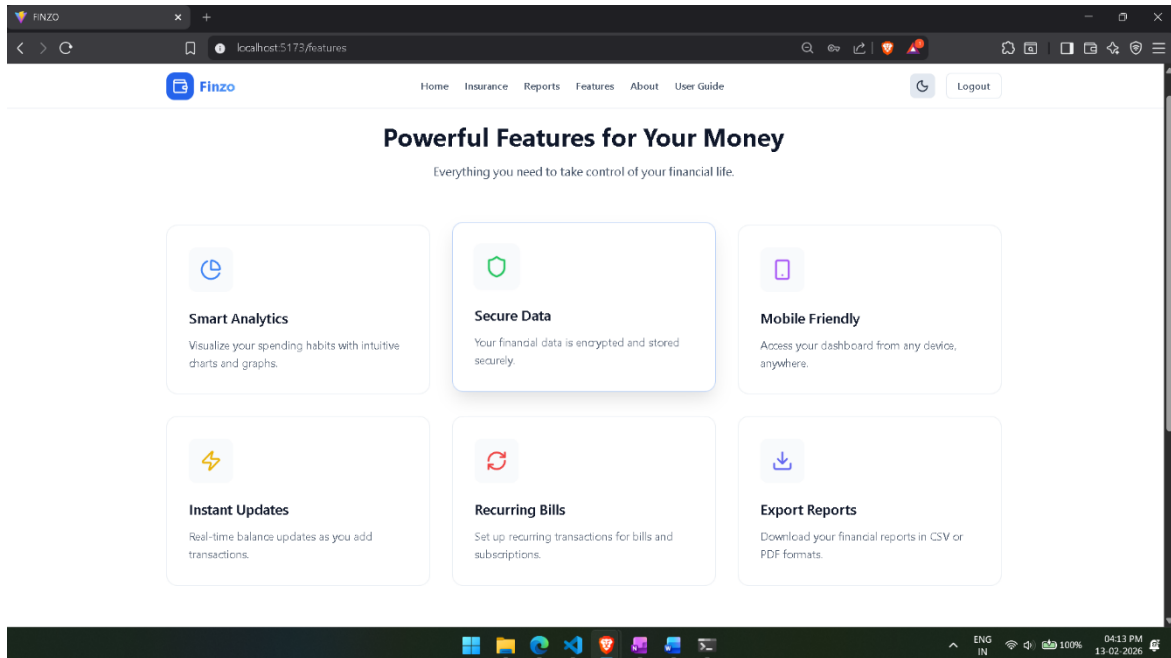
- Header:** 'HDFC - POL123456789' with a green heart icon.
- Details:** 'Health Insurance' and 'Coverage: ₹5,000,000'.
- Premium:** '₹12,000/yr'.
- Renewal:** '2030-01-01'.
- Actions:** 'Edit' and 'Delete' buttons.

The bottom of the image shows a Windows taskbar with various application icons and a system tray on the right displaying 'ENG IN', signal icons, '100%', and the date/time '04:12 PM 13-02-2026'.

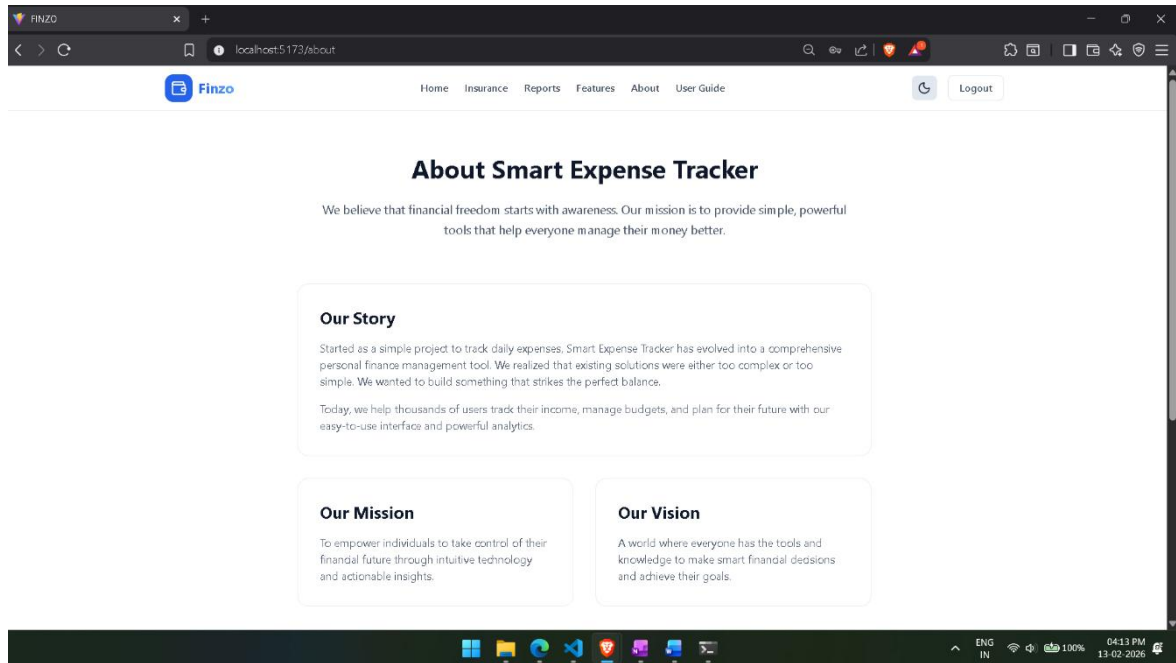
Reports & Analytics Page:



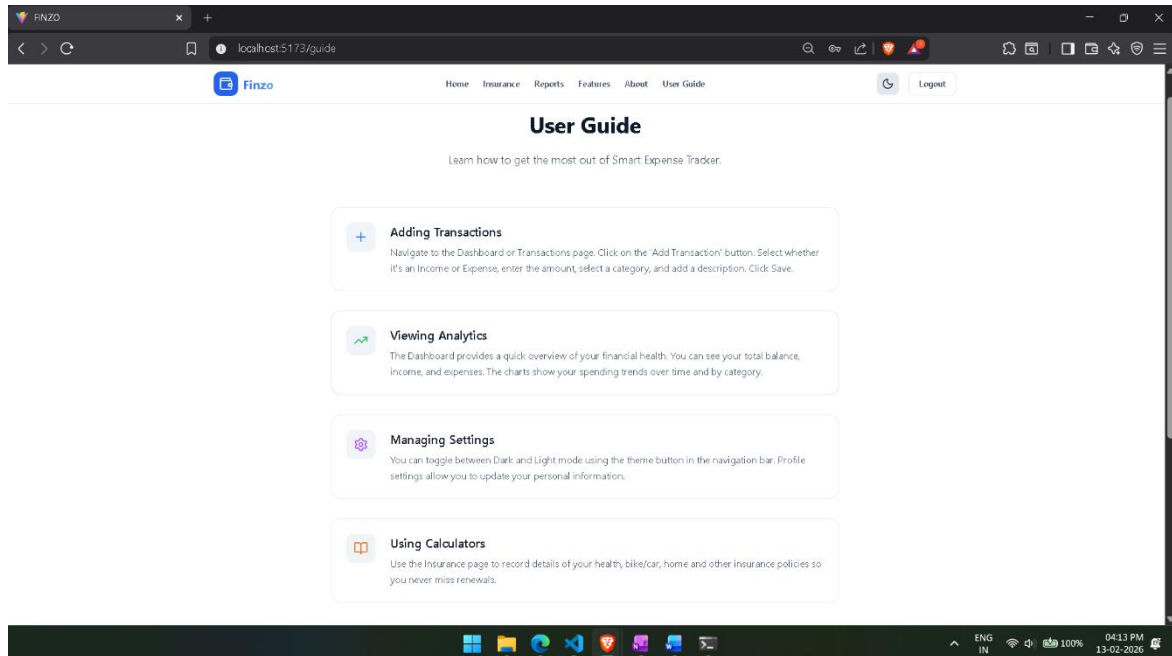
Features Page:



About Page:



User Guide Page:



3.8. Report Formats

The Smart Expenses Tracker system generates various reports to help users analyse their financial activities. These reports are presented in both tabular and graphical formats for better understanding and decision-making.

1. Daily Expense Report

- Displays all expenses recorded on a selected date.
- Includes:
 - Date
 - Category
 - Description
 - Amount
- Helps users track daily spending habits.

2. Monthly Expense Report

- Shows total expenses for a selected month.
- Includes:
 - Category-wise total spending
 - Overall monthly total
- Helps users analyse monthly financial patterns.

3. Category-wise Report

- Shows total spending under each category.
- Helps users identify high-expense categories.

4. Budget vs Actual Report

- Compares set budget with actual expenses.
- Displays:
 - Budget Amount
 - Actual Spending
 - Remaining Balance
- Helps users control overspending.

5. Graphical Reports

- Represented using charts (Bar Chart / Line Chart).
- Used for visual analysis of:
 - Category distribution
 - Monthly trends
 - Expense comparisons

Report Format Presentation

Reports are displayed in:

- **Tabular format** (for detailed records)
- **Graphical format** (using charts for visualization)

3.9. Test Procedures and Implementation

3.9.1 Test Procedures

Testing ensures that the system functions correctly across all layers of the architecture: Frontend (React), Backend (FastAPI), Authentication (JWT), and Database (SQLAlchemy with PostgreSQL/MySQL/SQLite). The testing strategy follows a multi-level approach including unit testing, integration testing, and end-to-end testing.

3.9.2 Testing Strategy

The application follows a layered testing approach:

Test Level	Purpose	Tools Used
Unit Testing	Test individual functions/components	Pytest, React Testing Library
Integration Testing	Test interaction between modules	Pytest + TestClient
API Testing	Validate backend endpoints	FastAPI TestClient / Postman
End-to-End Testing	Test complete user workflows	Cypress / Manual Testing
Database Testing	Validate DB operations	SQLAlchemy test DB

3.9.3 Backend Test Procedures (FastAPI)

1. Unit Testing

Unit tests validate:

- Business logic inside endpoint functions
- Database query logic
- Utility functions
- Authentication helpers (JWT validation)

Tool Used: pytest

Procedure:

1. Create a test database.
2. Override `get_db()` dependency.
3. Create mock test user.
4. Generate test JWT token.
5. Send request using `TestClient`.
6. Validate response status and structure.

2. Integration Testing

Integration tests verify:

- Router → Dependency → Database interaction
- JWT authentication workflow
- Data filtering by logged-in user

Test Flow:

1. Send authenticated request.
2. Verify JWT validation via `get_current_user`.
3. Confirm correct database filtering (`user_id == current_user.id`).
4. Validate serialized JSON response.

3. Authentication Testing

Tests verify:

- Valid JWT token → access granted
- Expired/invalid token → 401 Unauthorized
- No token → 401 Unauthorized.

3.9.4 Database Testing

A separate test database is used to prevent corruption of production data.

Procedure:

1. Create temporary test DB.
2. Use SQLAlchemy session override.
3. Insert mock records.
4. Perform CRUD operations.
5. Verify correct query results.
6. Rollback after test completion.

Test Cases:

- Insert transaction
- Update transaction
- Delete transaction
- Fetch transactions filtered by user

3.9.5 Frontend Test Procedures (React)

1. Component Testing

Tool Used: React Testing Library

Tests verify:

- Component renders correctly
- API call is triggered in use Effect
- Data is displayed in table
- Loading and error states render properly

2. API Client Testing

Tests verify:

- Axios base URL configuration
- JWT token added via interceptor
- Proper error handling on 401 responses

Mock local Storage to simulate token presence.

3.9.6 End-to-End Testing

End-to-end testing validates full system flow:

React UI → Axios → FastAPI → Database → JSON Response → UI Update

- User logs in.
- JWT token stored in local Storage.
- User navigates to Reports page.
- API call made with Bearer token.
- Backend validates user.
- Transactions returned.
- Table populated in UI.

Expected Result: Only logged-in user's transactions are displayed.

3.9.7 Test Environment

- Backend: FastAPI running locally
- Frontend: React development server
- Database: Separate test database
- Tools:
 - Pytest
 - FastAPI TestClient
 - React Testing Library
 - Postman (Manual API validation)

3.9.8 Test Case Summary

Module	Test Scenario	Expected Result
Login	Valid credentials	JWT returned
Login	Invalid credentials	401 error
Transactions	With valid token	200 + user data
Transactions	Without token	401 error
Reports UI	API success	Table populated
Reports UI	API failure	Error message displayed

3.9.9 Implementation

The system was tested across all architectural layers. Authentication, database filtering, API routing, and frontend rendering were validated. The tests confirmed:

- Secure JWT-based access control
- Correct database-user data isolation
- Accurate JSON serialization via Pydantic
- Proper frontend state management and UI updates

All critical functionalities performed as expected under both valid and invalid scenarios.

CHAPTER 4: USER MANUAL

4.1. User Manual

This chapter provides detailed guidance for end users and administrators on how to operate the system. The application is a web-based transaction management system developed using React (frontend) and FastAPI (backend) with JWT-based authentication and a relational database.

The system allows users to:

- Manage income and expenses
- Track insurance policies
- View financial reports
- Access analytics
- View system features and guidance
- Toggle theme settings
- Securely log in and log out

4.1.2 Starting the Application

1. Open a web browser.
2. Enter the application URL.
3. The Login page will be displayed.
4. Registered users can log in.
5. New users must register before accessing the system.

4.1.3 User Registration

1. Click on **Register**.
2. Fill in:
 - Username
 - Email
 - Password
3. Click **Submit**.
4. After successful registration, the system redirects to the login page.

4.1.5 User Login

1. Enter registered credentials.
2. Click **Login**.
3. If authentication is successful:
 - A JWT token is generated.
 - User is redirected to the Dashboard.
4. If authentication fails:
 - An error message is displayed.

4.1.6 Navigation Overview

After login, the main menu provides access to:

- **Home**
- **Insurance**
- **Reports**
- **Features**
- **About**
- **User Guide**
- Theme Toggle (Dark/Light mode)
- Logout Button

4.2. Operations Manual / Menu Explanation

4.2.1 Home (Dashboard)

The Home page displays:

- Financial overview
- Summary of income and expenses
- Real-time updates when transactions are added
- Charts and analytics (if implemented)

4.2.2 Insurance Page

The Insurance page allows users to manage their insurance policies.

Adding an Insurance Policy

1. Navigate to **Insurance**.
2. Fill in:
 - Provider / Company
 - Insurance Type (Health, Vehicle, etc.)
 - Policy Number
 - Annual Premium
 - Coverage Amount
 - Expiry / Renewal Date
3. Click **Save Policy**.

The policy will be stored and displayed under **Your Policies**.

Managing Policies

Each policy card includes:

- Policy details
- Premium amount
- Coverage amount
- Renewal date
- Edit option
- Delete option

Security:

Users can only access and manage their own policies.

4.2.3 Reports Page

The Reports page provides:

- List of transactions
- Categorized spending
- Financial summaries
- Structured data display

4.2.4 Features Page

The Features page highlights the core system capabilities:

- Smart Analytics
- Secure Data Storage
- Mobile Friendly Interface
- Instant Updates
- Recurring Bills (if implemented)

- Export Reports (CSV/PDF)

4.2.5 User Guide Page

The User Guide provides built-in instructions for:

- Adding Transactions
- Viewing Analytics
- Managing Settings
- Using Insurance Tracking

4.2.6 About Page

The About page includes:

- System background (Our Story)
- Mission statement
- Vision statement

4.2.7 Logout

- Removes JWT token from local storage.
- Ends session securely.
- Redirects to Login page.

4.3. Forms and Report Specifications

1. User Registration Form

Purpose:

Allows new users to create an account in the Finzo system.

Fields:

- Full Name
- Email Address
- Password
- Confirm Password

Validation:

- Email must be unique.
- Password must meet security requirements.
- All fields are mandatory.

Backend Handling:

- API Endpoint: `/api/auth/register`
- Data validated using **Pydantic schemas**.
- Stored in **User model** in the database.

Frontend Implementation:

- Located in `frontend/src/pages/Register.jsx`
- Uses **React form handling**
- Sends request using **Axios service layer**.

2. User Login Form

Purpose:

Authenticates registered users to access the system.

Fields:

- Email
- Password

Validation:

- Email must exist in the system.
- Password must match stored credentials.

Backend Handling:

- API Endpoint: /api/auth/login
- Uses **JWT authentication** for secure login.

Frontend Implementation:

- Located in frontend/src/pages/Login.jsx
- Stores JWT token using **React Context (AuthContext)**.

3. Transaction Entry Form

Purpose:

Allows users to record financial transactions.

Fields:

- Transaction Title
- Amount
- Transaction Type (Income / Expense)
- Category
- Date

- Notes (Optional)

Validation:

- Amount must be numeric.
- Required fields cannot be empty.

Backend Handling:

- API Endpoint: /api/transactions
- Data validated using **Transaction schema**.
- Stored in **Transaction model**.

Frontend Implementation:

- Located in frontend/src/pages/Dashboard.jsx or a dedicated transaction component.
- API calls made using **Axios services**.

4. Profile Update Form**Purpose:**

Allows users to update their personal information.

Fields:

- Full Name
- Email
- Password (optional update)

Backend Handling:

- API Endpoint: /api/users/profile
- Updates data in **User model**.

Frontend Implementation:

- Located in frontend/src/pages/Profile.jsx.

Report Specifications

1. Financial Summary Report

Purpose:

Displays an overview of a user's financial activity.

Data Included:

- Total Income
- Total Expenses
- Net Balance

Backend Source:

- Analytics API in backend/app/api/analytics
- Data aggregated using service layer logic.

Frontend Display:

- Implemented in Dashboard.
- Visualization using **Recharts**.

2. Transaction History Report

Purpose:

Provides a detailed list of all user transactions.

Fields Displayed:

- Transaction Title
- Category
- Amount
- Date
- Type (Income / Expense)

Backend Source:

- /api/transactions endpoint.

Frontend Display:

- Rendered using reusable **Card/Table components**.

3. Expense Analysis Report

Purpose:

Shows expense distribution across different categories.

Data Representation:

- Pie charts
- Bar graphs

Frontend Technology:

- **Recharts library** for data visualization.

Backend Source:

- Analytics service that aggregates transaction data by category.

4.4 Drawbacks and Limitations

Although the system is fully functional, the following limitations exist:

1. Limited role-based access (single user role).
2. JWT stored in local storage (security risk if XSS occurs).
3. No push notifications for insurance renewal.
4. Performance may decrease with very large datasets.

4.5 Proposed Enhancements

Future improvements may include:

- Email/SMS reminders for insurance renewals.
- Advanced graphical analytics and dashboards.
- Role-based authentication (Admin/User).
- Two-factor authentication (2FA).
- Cloud deployment with scalable architecture.
- Dedicated mobile application.
- Budget planning and goal tracking features.
- Auto-renewal alerts and recurring transaction automation.
- Integration with banking APIs.
- Enhanced data encryption and refresh token mechanism.

CHAPTER 5: CONCLUSIONS

The Smart Expenses Tracker has been successfully designed and implemented to provide an efficient and reliable solution for managing personal financial expenses. The application overcomes the limitations of traditional manual record-keeping methods by offering a secure, automated, and user-friendly digital platform for tracking daily financial transactions.

The system allows users to record, categorize, and analyse expenses in a structured manner. Features such as budget management, graphical reports, category-wise summaries, and insurance policy tracking provide valuable insights into financial activities and support informed decision-making. The integration of modern technologies including React, FastAPI, and MySQL ensures high performance, scalability, and secure data management.

The system follows a modular architecture separating the frontend, backend, and database layers, which enhances maintainability and future scalability. Secure authentication using JWT tokens and encrypted passwords ensures data confidentiality and protection.

Overall, the Smart Expenses Tracker successfully achieves its objective of simplifying expense management and improving financial awareness. The project also provides a strong foundation for future enhancements such as mobile integration, machine learning-based expense prediction, cloud deployment, automated notifications, and advanced analytics.

In conclusion, this project demonstrates the effective application of modern web technologies in developing a practical, scalable, and real-world financial management system.

6. Bibliography

1. OpenAI

ChatGPT: AI Language Model for Software Development Assistance,

Available: <https://chatgpt.com/>

2. Google

Google Search Engine,

Available: google.com.

3. YouTube

Educational Programming and Web Development Tutorials,

Available: <https://www.youtube.com>.

4. Google Antigravity

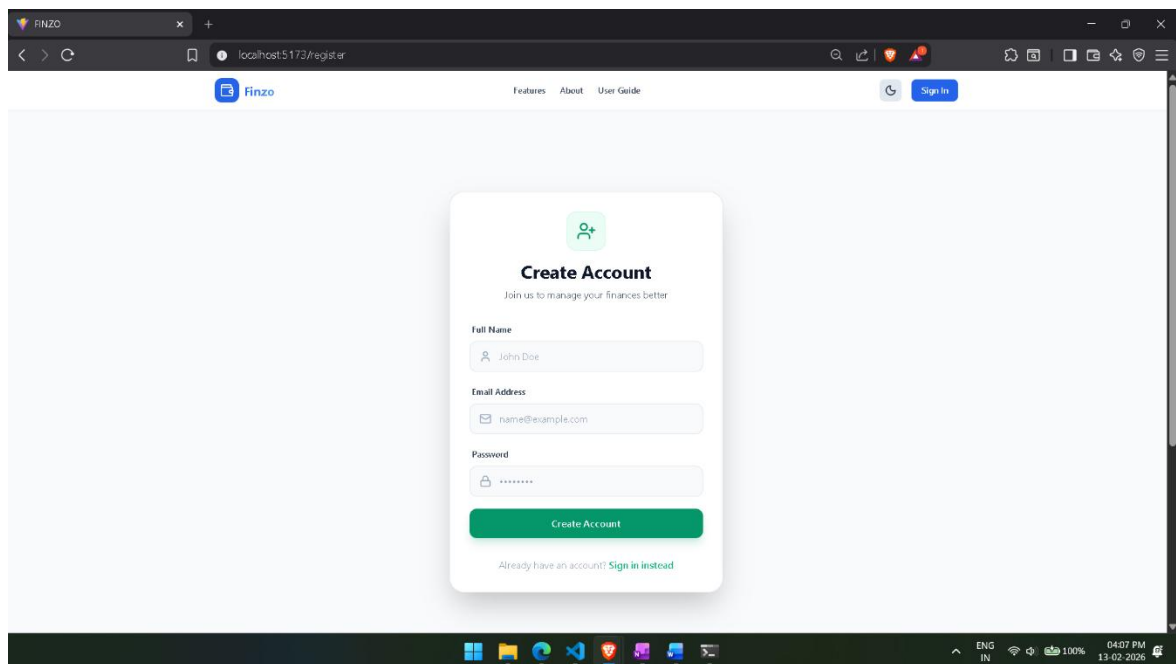
Google Antigravity AI Tool,

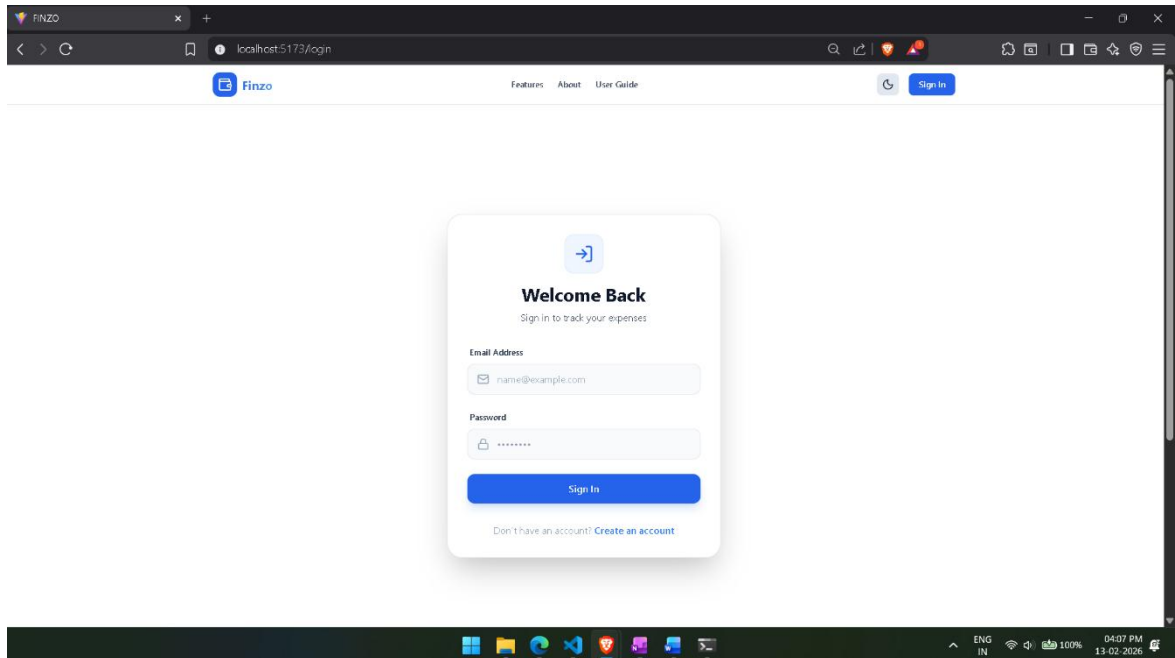
Available: <https://antigravity.google.com> Google Antigravity Download.

7. ANNEXURES

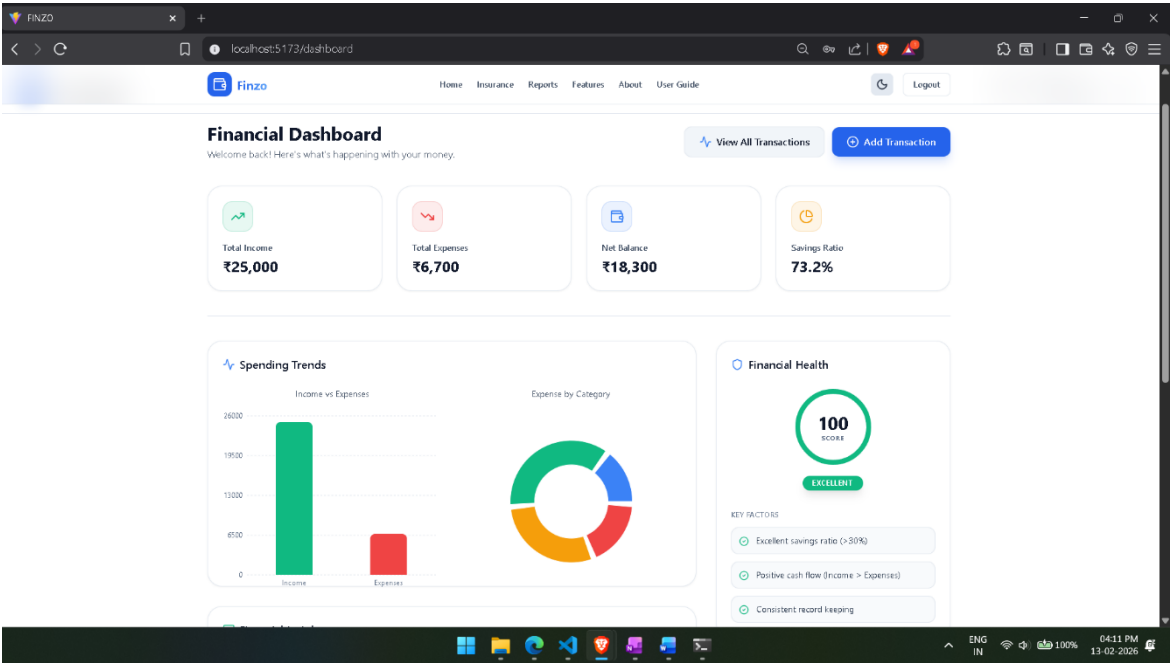
7.1. ANNEXURE 1: USER INTERFACE SCREENS

Register Page:

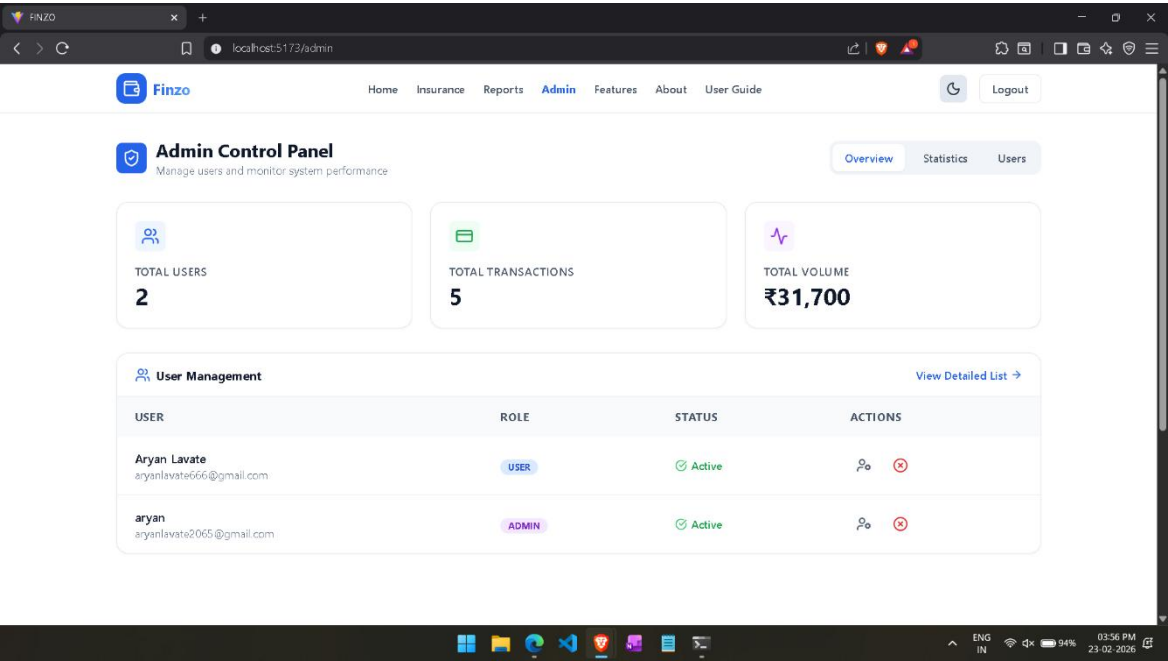


Login Page:

Home Page:



Admin Page:



7.2. ANNEXURE 3: SAMPLE PROGRAM CODE

Home Page:

```
import React, { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/useAuth';
import { UserPlus, Mail, Lock, User, AlertCircle, CheckCircle2 } from 'lucide-react';

const Register = ({ onToggleMode }) => {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [fullName, setFullName] = useState("");
  const [error, setError] = useState("");
  const [success, setSuccess] = useState(false);
  const [loading, setLoading] = useState(false);
  const { register } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);
    try {
      await register(email, password, fullName);
      setSuccess(true);
      // If an onToggleMode callback is provided (e.g. from an auth modal),
      // use it to switch back to login, otherwise navigate to the login page.
      setTimeout(() => {
        if (typeof onToggleMode === 'function') {
          onToggleMode();
        } else {
          navigate('/login');
        }
      }, 2000);
    } catch (error) {
      setError(error.message);
    }
  };
};
```

```

    }
    }, 1500);
  } catch (err) {
    setError(err.response?.data?.detail || 'Registration failed. Please try again.');
```

} finally {

```

    setLoading(false);
  }
};

return (
  <div className="min-h-screen flex items-center justify-center p-4 bg-slate-50 dark:bg-gradient-to-br dark:from-slate-950 dark:via-slate-900 dark:to-slate-950">

    <div className="w-full max-w-md bg-white dark:bg-slate-900/50 backdrop-blur-xl border border-slate-200 dark:border-slate-800 rounded-3xl p-8 shadow-2xl">

      <div className="flex flex-col items-center mb-8">

        <div className="w-16 h-16 bg-emerald-50 dark:bg-emerald-600/20 rounded-2xl flex items-center justify-center mb-4 border border-emerald-100 dark:border-emerald-500/30">

          <UserPlus className="w-8 h-8 text-emerald-600 dark:text-emerald-500" />

        </div>

        <h1 className="text-3xl font-bold text-slate-900 dark:text-white">Create Account</h1>

        <p className="text-slate-600 dark:text-slate-400 mt-2">Join us to manage your finances better</p>

      </div>

      {error && (
        <div className="mb-6 p-4 bg-red-500/10 border border-red-500/20 rounded-xl flex items-center gap-3 text-red-400 text-sm">

          <AlertCircle className="w-5 h-5 flex-shrink-0" />

          <p>{error}</p>

        </div>

      )}

      {success && (
        <div className="mb-6 p-4 bg-emerald-500/10 border border-emerald-500/20 rounded-xl flex items-center gap-3 text-emerald-400 text-sm">

          <CheckCircle2 className="w-5 h-5 flex-shrink-0" />

          <p>Registration successful! Redirecting to login...</p>

        </div>

```

```

    })

    <form onSubmit={handleSubmit} className="space-y-5">
      <div className="space-y-2">
        <label className="text-sm font-medium text-slate-700 dark:text-slate-300 ml-1">Full Name</label>
        <div className="relative">
          <User className="absolute left-4 top-1/2 -translate-y-1/2 w-5 h-5 text-slate-400 dark:text-slate-500"
        />
          <input
            type="text"
            required
            value={fullName}
            onChange={(e) => setFullName(e.target.value)}
            className="w-full bg-slate-50 dark:bg-slate-950 border border-slate-200 dark:border-slate-800
rounded-xl py-3 pl-12 pr-4 text-slate-900 dark:text-white placeholder:text-slate-400 dark:placeholder:text-slate-600
focus:outline-none focus:ring-2 focus:ring-emerald-500/50 focus:border-emerald-500 transition-all"
            placeholder="John Doe"
          />
        </div>
      </div>

      <div className="space-y-2">
        <label className="text-sm font-medium text-slate-700 dark:text-slate-300 ml-1">Email Address</label>
        <div className="relative">
          <Mail className="absolute left-4 top-1/2 -translate-y-1/2 w-5 h-5 text-slate-400 dark:text-slate-500"
        />
          <input
            type="email"
            required
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            className="w-full bg-slate-50 dark:bg-slate-950 border border-slate-200 dark:border-slate-800
rounded-xl py-3 pl-12 pr-4 text-slate-900 dark:text-white placeholder:text-slate-400 dark:placeholder:text-slate-600
focus:outline-none focus:ring-2 focus:ring-emerald-500/50 focus:border-emerald-500 transition-all"
            placeholder="name@example.com"
          />
        </div>
      </div>
    </form>
  )
}

```

```

</div>

<div className="space-y-2">
  <label className="text-sm font-medium text-slate-700 dark:text-slate-300 ml-1">Password</label>
  <div className="relative">
    <Lock className="absolute left-4 top-1/2 -translate-y-1/2 w-5 h-5 text-slate-400 dark:text-slate-500"
  />

    <input
      type="password"
      required
      value={password}
      onChange={(e) => setPassword(e.target.value)}

      className="w-full bg-slate-50 dark:bg-slate-950 border border-slate-200 dark:border-slate-800
rounded-xl py-3 pl-12 pr-4 text-slate-900 dark:text-white placeholder:text-slate-400 dark:placeholder:text-slate-600
focus:outline-none focus:ring-2 focus:ring-emerald-500/50 focus:border-emerald-500 transition-all"

      placeholder="••••••••"
    />
  </div>
</div>

<div>

  <button
    type="submit"

    disabled={loading || success}

    className="w-full bg-emerald-600 hover:bg-emerald-500 disabled:bg-emerald-600/50 text-white font-
semibold py-3 rounded-xl shadow-lg shadow-emerald-600/20 transition-all active:scale-[0.98] hover:shadow-emerald-
500/40"

    >

      {loading ? 'Creating account...' : 'Create Account'}

    </button>
  </form>

  <div className="mt-8 text-center">

    <p className="text-slate-400">

      Already have an account?{' '}

    <Link
      to="/login"

      className="text-emerald-500 hover:text-emerald-400 font-medium transition-colors"

```

```
        >  
        Sign in instead  
      </Link>  
    </p>  
  </div>  
</div>  
</div>  
);  
};  
  
export default Register;
```

Login Page:

```

import React, { useState } from 'react';
import { Link } from 'react-router-dom';
import { ArrowRight, CheckCircle2 } from 'lucide-react';
// Default hero image (exists in repo). You can replace this file with your Canva image.
import heroImage from '../assets/hero-image.png';

const LandingPage = () => {
  // Prefer the Canva image if present in `frontend/public/` with the exact filename.
  // If it's missing, fall back to the bundled hero image so Vite never crashes.
  const [heroSrc, setHeroSrc] = useState(
    'https://marketplace.canva.com/EAFYsaCVVjE/1/0/800w/canva-white-and-purple-simple-daily-expense-pie-chart-JU_4nqnx7RY.jpg'
  );

  return (
    <div className="flex flex-col">
      {/* Hero Section */}
      <section className="pt-20 pb-32 px-4 sm:px-6 lg:px-8 max-w-7xl mx-auto flex flex-col lg:flex-row items-center gap-12">
        <div className="flex-1 text-center lg:text-left">
          <h1 className="text-4xl sm:text-5xl lg:text-6xl font-bold tracking-tight text-slate-900 dark:text-white mb-6">
            One smart app to manage all your <span className="text-blue-600 dark:text-brand-secondary">accounts, budgets, and bills.</span>
          </h1>
          <p className="text-lg sm:text-xl text-slate-600 dark:text-slate-300 mb-8 max-w-2xl mx-auto lg:mx-0">
            Your all-in-one expense manager to track income, expenses, and savings with ease.
          </p>
          <div className="flex flex-col sm:flex-row items-center justify-center lg:justify-start gap-4">
            <Link
              to="/register"
              className="px-8 py-4 bg-blue-600 hover:bg-blue-500 dark:bg-brand-primary dark:hover:bg-brand-accent text-white text-lg font-semibold rounded-xl shadow-lg shadow-blue-600/20 dark:shadow-brand-primary/20 transition-all hover:scale-105 flex items-center gap-2"
            >

```

```

        Get Started Free <ArrowRight className="w-5 h-5" />
      </Link>
    <Link
      to="/features"
      className="px-8 py-4 bg-slate-100 dark:bg-slate-800 hover:bg-slate-200 dark:hover:bg-slate-700 text-slate-900 dark:text-white text-lg font-semibold rounded-xl transition-all"
    >
      View Features
    </Link>
  </div>
  <div className="mt-8 flex items-center justify-center lg:justify-start gap-6 text-sm text-slate-500 dark:text-slate-400">
    <div className="flex items-center gap-2">
      <CheckCircle2 className="w-4 h-4 text-green-500" />
      <span>Free Forever</span>
    </div>
    <div className="flex items-center gap-2">
      <CheckCircle2 className="w-4 h-4 text-green-500" />
      <span>No Credit Card</span>
    </div>
  </div>
  <div>
    <div className="flex-1 relative">
      <div className="absolute -inset-4 bg-gradient-to-r from-blue-600 via-purple-600 to-pink-500 dark:from-brand-primary dark:via-brand-accent dark:to-brand-secondary rounded-3xl blur-2xl opacity-30 animate-pulse" />
      <img
        src={heroSrc}
        onError={() => setHeroSrc(heroImage)}
        alt="Daily Expense Overview"
        className="relative rounded-3xl shadow-2xl border border-slate-200 dark:border-slate-800 transform hover:-translate-y-2 hover:shadow-[0_20px_60px_rgba(59,130,246,0.45)] dark:hover:shadow-[0_20px_60px_rgba(131,140,229,0.45)] transition-all duration-500"
      />
    </div>
  </section>
</div>
);

```

```
};
```

```
export default LandingPage;
```

Database Connection:

```

import pymysql

from sqlalchemy import create_engine
from sqlalchemy.exc import OperationalError

# The URL currently in your app
DB_URL = "mysql+pymysql://root:aryan0726@localhost:3306/expense_tracker"

def check_connection():
    print(f"Testing connection to: {DB_URL}")

    try:
        # 1. Try raw connection first
        print("Attempting raw pymysql connection...")

        conn = pymysql.connect(
            user="root",
            password="aryan0726",
            host="localhost",
            port=3306,
            database="expense_tracker"
        )
        conn.close()
        print("SUCCESS: Raw connection successful!")

        # 2. Try SQLAlchemy connection
        print("Attempting SQLAlchemy connection...")
        engine = create_engine(DB_URL)
        with engine.connect() as connection:
            print("SUCCESS: SQLAlchemy connection successful!")

    except pymysql.err.OperationalError as e:
        print("\n CONNECTION FAILED")
        print(f"Error details: {e}")

```

```
print("\nDiagnosis:")

if e.args[0] == 2003: # Can't connect to MySQL server
    print("1. Is MySQL Server running?")
    print("2. Is it listening on port 3306?")
elif e.args[0] == 1045: # Access denied
    print("1. Password 'aryan0726' for user 'root' is incorrect.")
elif e.args[0] == 1049: # Unknown database
    print("1. Database 'expense_tracker' does not exist.")

except Exception as e:
    print(f"\n UNEXPECTED ERROR: {e}")

if __name__ == "__main__":
    check_connection()
```

Admin Page:

```

import React, { useEffect, useState } from 'react';

import { Link, useNavigate } from 'react-router-dom';

import api from '../services/api';

import { Users, CreditCard, Activity, ArrowRight, ShieldCheck, UserCog, CheckCircle, XCircle, Trash2,
LayoutDashboard, Settings } from 'lucide-react';

const AdminDashboard = ({ activeTab = 'all' }) => {

  const [stats, setStats] = useState(null);

  const [users, setUsers] = useState([]);

  const [loading, setLoading] = useState(true);

  const [error, setError] = useState(null);

  const [actionLoading, setActionLoading] = useState(null);

  const navigate = useNavigate();

  const fetchData = async () => {

    try {

      const [statsRes, usersRes] = await Promise.all([

        api.get('/admin/stats'),

        api.get('/admin/users')

      ]);

      setStats(statsRes.data);

      setUsers(usersRes.data);

    } catch (err) {

      setError('Failed to fetch admin data');

      console.error(err);

    } finally {

      setLoading(false);

    }

  };

  useEffect(() => {

    fetchData();

  }, []);

```

```

const toggleStatus = async (userId) => {
  setActionLoading(userId);

  try {
    await api.put(`/admin/users/${userId}/toggle-status`);

    await fetchData(); // Refresh data
  } catch (err) {
    alert(err.response?.data?.detail || 'Failed to toggle status');
  } finally {
    setActionLoading(null);
  }
};

const changeRole = async (userId, newRole) => {
  setActionLoading(userId);

  try {
    await api.put(`/admin/users/${userId}/role?role=${newRole}`);

    await fetchData(); // Refresh data
  } catch (err) {
    alert(err.response?.data?.detail || 'Failed to change role');
  } finally {
    setActionLoading(null);
  }
};

if (loading) {
  return (
    <div className="flex items-center justify-center min-h-[60vh]">
      <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div>
    </div>
  );
}

if (error) {
  return (

```

```

<div className="max-w-7xl mx-auto px-4 py-12 text-center">
  <div className="bg-red-50 dark:bg-red-900/20 p-6 rounded-2xl inline-block">
    <p className="text-red-600 dark:text-red-400 font-medium">{error}</p>
  </div>
</div>

);
}

const showStats = activeTab === 'all' || activeTab === 'stats';
const showUsers = activeTab === 'all' || activeTab === 'users';

return (
  <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
    <div className="flex flex-col md:flex-row md:items-center justify-between gap-4 mb-8">
      <div className="flex items-center gap-3">
        <div className="bg-blue-600 p-2 rounded-lg">
          <ShieldCheck className="w-6 h-6 text-white" />
        </div>
        <div>
          <h1 className="text-2xl font-bold text-slate-900 dark:text-white">Admin Control Panel</h1>
          <p className="text-slate-500 dark:text-slate-400 text-sm">Manage users and monitor system
performance</p>
        </div>
      </div>
    </div>

    <div className="flex items-center gap-2 bg-slate-100 dark:bg-slate-800 p-1 rounded-xl">
      <button
        onClick={() => navigate('/admin')}
        className={`px-4 py-2 rounded-lg text-sm font-semibold transition-all ${activeTab === 'all'
          ? 'bg-white dark:bg-slate-700 text-blue-600 dark:text-white shadow-sm'
          : 'text-slate-600 dark:text-slate-400 hover:text-slate-900 dark:hover:text-white'
        }`>
        Overview
      </button>

```

```

<button
  onClick={() => navigate('/admin/stats')}
  className={`px-4 py-2 rounded-lg text-sm font-semibold transition-all ${activeTab === 'stats'
    ? 'bg-white dark:bg-slate-700 text-blue-600 dark:text-white shadow-sm'
    : 'text-slate-600 dark:text-slate-400 hover:text-slate-900 dark:hover:text-white'
  }}
>
  Statistics
</button>

<button
  onClick={() => navigate('/admin/users')}
  className={`px-4 py-2 rounded-lg text-sm font-semibold transition-all ${activeTab === 'users'
    ? 'bg-white dark:bg-slate-700 text-blue-600 dark:text-white shadow-sm'
    : 'text-slate-600 dark:text-slate-400 hover:text-slate-900 dark:hover:text-white'
  }}
>
  Users
</button>
</div>
</div>

{/* Stats Overview */}
{showStats && (
  <div className="grid grid-cols-1 md:grid-cols-3 gap-6 mb-8">
    <div className="bg-white dark:bg-slate-900 p-6 rounded-2xl border border-slate-200 dark:border-slate-800
shadow-sm transition-all hover:shadow-md">
      <div className="flex items-center justify-between mb-4">
        <div className="p-2 bg-blue-50 dark:bg-blue-900/20 rounded-lg text-blue-600 dark:text-blue-400">
          <Users className="w-6 h-6" />
        </div>
      </div>
      <p className="text-sm text-slate-600 dark:text-slate-400 font-medium uppercase tracking-wider">Total
Users</p>
      <h3 className="text-3xl font-bold text-slate-900 dark:text-white mt-1">{stats.total_users}</h3>
    </div>
  </div>
)

```

```

<div className="bg-white dark:bg-slate-900 p-6 rounded-2xl border border-slate-200 dark:border-slate-800
shadow-sm transition-all hover:shadow-md">

  <div className="flex items-center justify-between mb-4">

    <div className="p-2 bg-green-50 dark:bg-green-900/20 rounded-lg text-green-600 dark:text-green-
400">

      <CreditCard className="w-6 h-6" />

    </div>

  </div>

  <p className="text-sm text-slate-600 dark:text-slate-400 font-medium uppercase tracking-wider">Total
Transactions</p>

  <h3 className="text-3xl font-bold text-slate-900 dark:text-white mt-1">{stats.total_transactions}</h3>

</div>

<div className="bg-white dark:bg-slate-900 p-6 rounded-2xl border border-slate-200 dark:border-slate-800
shadow-sm transition-all hover:shadow-md">

  <div className="flex items-center justify-between mb-4">

    <div className="p-2 bg-purple-50 dark:bg-purple-900/20 rounded-lg text-purple-600 dark:text-purple-
400">

      <Activity className="w-6 h-6" />

    </div>

  </div>

  <p className="text-sm text-slate-600 dark:text-slate-400 font-medium uppercase tracking-wider">Total
Volume</p>

  <h3 className="text-3xl font-bold text-slate-900 dark:text-white mt-1">₹{stats.total_volume.toLocaleString()}</h3>

</div>

</div>

)}

{/* User Management Section */}

{showUsers && (

  <div className="bg-white dark:bg-slate-900 rounded-2xl border border-slate-200 dark:border-slate-800
overflow-hidden shadow-sm">

    <div className="px-6 py-4 border-b border-slate-200 dark:border-slate-800 flex items-center justify-
between">

      <h2 className="font-bold text-slate-900 dark:text-white flex items-center gap-2">

        <Users className="w-5 h-5 text-blue-600" />

        User Management

      </h2>

```

```

{activeTab === 'all' && (
  <button
    onClick={() => navigate('/admin/users')}
    className="text-sm text-blue-600 dark:text-blue-400 hover:underline flex items-center gap-1 font-
medium"
  >
    View Detailed List <ArrowRight className="w-4 h-4" />
  </button>
)}
</div>

<div className="overflow-x-auto">
  <table className="w-full text-left">
    <thead className="bg-slate-50 dark:bg-slate-800/50 text-slate-600 dark:text-slate-400 text-sm font-
semibold uppercase tracking-wider">
      <tr>
        <th className="px-6 py-4">User</th>
        <th className="px-6 py-4">Role</th>
        <th className="px-6 py-4">Status</th>
        <th className="px-6 py-4">Actions</th>
      </tr>
    </thead>
    <tbody className="divide-y divide-slate-200 dark:divide-slate-800">
      {users.map((user) => (
        <tr key={user.id} className="hover:bg-slate-50 dark:hover:bg-slate-800/30 transition-colors">
          <td className="px-6 py-4">
            <div className="flex flex-col">
              <span className="text-slate-900 dark:text-slate-200 font-
semibold">{user.full_name}</span>
              <span className="text-slate-500 dark:text-slate-400 text-sm">{user.email}</span>
            </div>
          </td>
          <td className="px-6 py-4">
            <span className={`inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-bold
uppercase ${user.role === 'admin'
? 'bg-purple-100 text-purple-700 dark:bg-purple-900/30 dark:text-purple-400'
: 'bg-blue-100 text-blue-700 dark:bg-blue-900/30 dark:text-blue-400'

```

```

    }}>
    {user.role}
  </span>
</td>
<td className="px-6 py-4">
  <div className="flex items-center gap-1.5">
    {user.is_active ? (
      <img alt="Green checkmark icon" data-bbox="338 254 356 264"/>
      <CheckCircle className="w-4 h-4 text-green-500" />
      <span className="text-green-600 dark:text-green-400 text-sm font-
medium">Active</span>
    ) : (
      <img alt="Red X icon" data-bbox="338 386 356 396"/>
      <XCircle className="w-4 h-4 text-red-500" />
      <span className="text-red-600 dark:text-red-400 text-sm font-
medium">Deactivated</span>
    )}
  </div>
</td>
<td className="px-6 py-4">
  <div className="flex items-center gap-3">
    <button
      onClick={() => changeRole(user.id, user.role === 'admin' ? 'user' : 'admin')}
      disabled={actionLoading === user.id}
      className="p-2 text-slate-600 dark:text-slate-400 hover:text-blue-600 dark:hover:text-
brand-accent transition-colors disabled:opacity-50"
      title={user.role === 'admin' ? "Make User" : "Make Admin"}
    >
      <UserCog className="w-5 h-5" />
    </button>
    <button
      onClick={() => toggleStatus(user.id)}
      disabled={actionLoading === user.id}
      className={`p-2 transition-colors disabled:opacity-50 ${user.is_active

```

```

        ? 'text-red-600 hover:text-red-700'
        : 'text-green-600 hover:text-green-700'
      }}
      title={user.is_active ? "Deactivate User" : "Activate User"}
    >
      {user.is_active ? <XCircle className="w-5 h-5" /> : <CheckCircle className="w-5 h-
5" />}
    </button>
  </div>
</td>
</tr>
)}}
</tbody>
</table>
</div>
</div>
)}
</div>
);
};

export default AdminDashboard;

```

